Checkout Incident Detection Agent Problem Statement

Background

E-commerce teams monitor checkout activity 🛒 to catch issues that can **impact revenue** (especially payment failures). At scale, hundreds of checkout events may arrive every hour, and it's not practical to manually triage each one.
We want an agent that can consistently review new events, decide whether the situation warrants attention, and publish a clean "command" for downstream execution.

Goal

Build an **E-commerce Operations Agent** in Zapier that reads new checkout events from Google Sheets, evaluates their severity using baseline metrics, and outputs **exactly one command** per run (ALERT or LOG). The agent **does not** execute fixes; it only publishes intent via a webhook and updates state.

Data Architecture

You are required to set up a Google Spreadsheet with the following four distinct sheets to simulate the database.

1. **Cart Events Sheet (The Input)**
   The agent needs all the raw data for every user's checkout, including when it happened, what stage it failed at, how much the cart was worth, and what device was used, for it to have the facts it needs to make an initial decision.

   - **Columns: timestamp**, **checkout_step** (Shipping/Payment/Address), **cart_value, device** (Mobile/Desktop).

A	B	C	D
timestamp	**checkout_step**	**cart_value**	**device**
2026-01-28 13:45:00	PAYMENT	4599	MOBILE
2026-01-28 14:10:00	ADDRESS	30	MOBILE
2026-01-28 14:15:00	SHIPPING	1199	DESKTOP

2. **Baseline Metrics Sheet (The Context)**
   The agent needs to know the *normal* rate of failure for each checkout step for it to decide if a new issue is a truly critical problem or just part of the everyday baseline noise.

   - **Columns: checkout_step, baseline_rate.**
   - **Usage***:* The agent uses this to judge if a current failure is unusual.

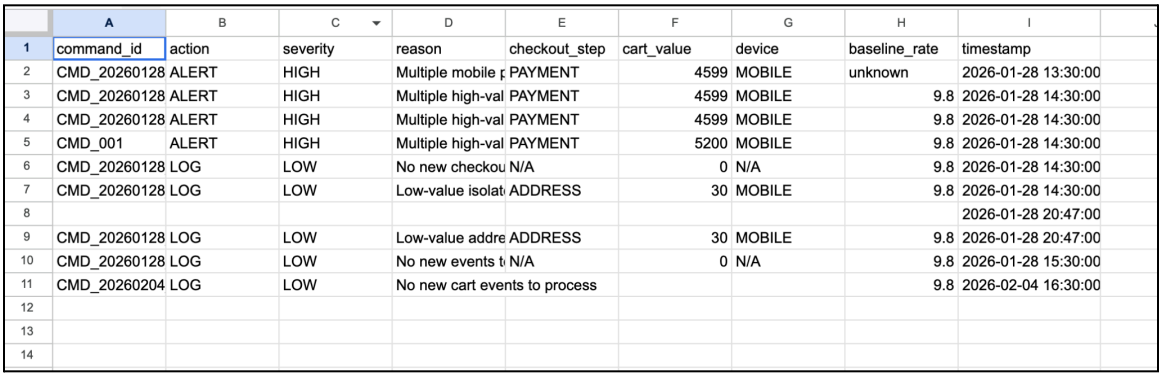
A	B
checkout_step	**baseline_rate**
PAYMENT	9.8
ADDRESS	3.2
SHIPPING	5.6

3. **State Sheet (The Memory)**
   The agent needs a way to record the last event it successfully processed for it to remember where it left off, ensuring it only processes new events and doesn't create duplicate alerts in its next run.
   - **Columns: key, value.**
   - ***Required*** *Key:* **last_processed_timestamp.**

	A	B	C
1	**key**	**value**	
2	last_processed_time	2026-01-28 15:17:00	
3			
4			

4. **Agent Sheet (The Output)**
   The agent needs a sheet where it can formally write its final decision, the command to either **ALERT** or **LOG**, for the system to know exactly what action to take.

   - **Columns: command_id, action, severity, reason, checkout_step, cart_value, device, baseline_rate, timestamp**



	A	B	C	D	E	F	G	H	I
1	command_id	action	severity	reason	checkout_step	cart_value	device	baseline_rate	timestamp
2	CMD_20260128	ALERT	HIGH	Multiple mobile p	PAYMENT	4599	MOBILE	unknown	2026-01-28 13:30:00
3	CMD_20260128	ALERT	HIGH	Multiple high-val	PAYMENT	4599	MOBILE	9.8	2026-01-28 14:30:00
4	CMD_20260128	ALERT	HIGH	Multiple high-val	PAYMENT	4599	MOBILE	9.8	2026-01-28 14:30:00
5	CMD_001	ALERT	HIGH	Multiple high-val	PAYMENT	5200	MOBILE	9.8	2026-01-28 14:30:00
6	CMD_20260128	LOG	LOW	No new checkou	N/A	0	N/A	9.8	2026-01-28 14:30:00
7	CMD_20260128	LOG	LOW	Low-value isolat	ADDRESS	30	MOBILE	9.8	2026-01-28 14:30:00
8									2026-01-28 20:47:00
9	CMD_20260128	LOG	LOW	Low-value addre	ADDRESS	30	MOBILE	9.8	2026-01-28 20:47:00
10	CMD_20260128	LOG	LOW	No new events t	N/A	0	N/A	9.8	2026-01-28 15:30:00
11	CMD_20260204	LOG	LOW	No new cart events to process				9.8	2026-02-04 16:30:00
12									
13									
14									

Functional Requirements

1. **State Management (Critical)**
   a. The agent must **read** the **last_processed_timestamp** from the **state** sheet at the start of every run.
   b. It must **filter** the **cart_events** to only process rows where the timestamp is *newer* than the stored state.
   c. At the end of the run, it must **update** the **state** sheet with the timestamp of the latest processed event.

2. **Decision Logic**
   The agent must evaluate every new event based on these business rules:

   a. **Severity High:** If the issue occurs during the **PAYMENT** step (critical revenue impact).
   b. **Severity High:** If the **cart_value** is significantly high.
   c. **Severity High:** If the failure rate exceeds the **baseline_rate**.
   d. **Severity High:** If multiple mobile failures occur (potential app bug).
   e. Action:
      i. **ALERT:** If severity is HIGH.
      ii. **LOG:** If severity is LOW (e.g., low value, isolated incident).

3. **Execution & Hand-off**
   a. The agent must write exactly **one row** to the **agent_commands** sheet per decision.

b. **Mandatory:** The agent must trigger a **Webhook (POST request)** to broadcast its decision to downstream systems.
c. The Webhook JSON body must contain all 9 fields from the command row (action, severity, reason, etc.).

Functional Requirements

1. **Zero Duplicates**: The agent never processes the same checkout event row twice (State Management works).
2. **Smart Triage:** The agent correctly identifies high-value Payment failures as **HIGH/ALERT**
3. **Connectivity:** The Webhook fires successfully with a valid JSON payload for every decision made.

Note: The columns and sheets listed are just for examples. Feel free to add or refine details based on their specific e-commerce data and requirements.